Deston Willis

Dr. Forouraghi

Independent Research Project

1 April 2026

Report 2

I started by changing my Streamlit app so that it takes files as the input given to the model instead of text. I did this by removing the previous functionality I had for uploading directories by entering in its file path and replacing it with a file uploader that only allows .java files to be uploaded. When a file is uploaded, a message with the name of the file is displayed in the chat along with a button that lets the user start the code evaluation. I also added a reset button in the sidebar to clear the chat.

Code for the Streamlit Interface:

```python
# Streamlit Interface
st.set_page_config(page_title="AI Grading Bot", layout="wide")
st.title("AI Grading Assistant")

# Create chat history
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []
# Clears the chat history
def reset_button():  1 usage  new *
    st.session_state.chat_history.clear()
# Tells the model to run and generate a response
def run_model_button():  1 usage  new *
    with st.spinner("Executing Local Workflow..."):
        result = app.invoke({"messages": [HumanMessage(content=file_input.read().decode("utf-8"))]})
        # Displays the output from the code and the LLM's response to it
        final_answer = "Output:" + "\n\n" + result["output"].content + "\n\n" + result["messages"][-1].content
        st.session_state.chat_history.append(AIMessage(final_answer))

# Sidebar for Reset button
with st.sidebar:
    st.title("Reset Chat History")
    st.button( label: "Reset", on_click= reset_button)

# Chat Window
# Creates a place for users to upload files and a button to start the model's evaluation
file_input = st.file_uploader(label="Please Upload a Java File to be evaluated", type=["java"])
if file_input:
    st.session_state.chat_history.append(AIMessage("Files Uploaded: " + "\n\n" + file_input.name))
    st.button( label: "Evaluate Code", on_click=run_model_button)

# Display the conversation
for msg in st.session_state.chat_history:
    role = "user" if isinstance(msg, HumanMessage) else "assistant"
    with st.chat_message(role):
        st.markdown(msg.content)
```

Next I created a tool to serve as the Java Compiler. I initially researched different tools that I could import into my program but ultimately decided to define a tool in my program instead. This tool creates a temporary directory to run the code. This allows for the code to be in an isolated directory so that the code can't access host system files. The tool then inserts the code into a file in the temporary directory. It then compiles the code. If the code is compiled successfully, then it is run and the output or error message is returned. If the code isn't compiled successfully, then the error message is returned. To make this tool work, I had to download a JDK and add it in the project directory so that the tool could successfully compile and run the

code. I then connected the tool to the LLM that I decided to use, which is llama3.1:8b since it produced the best results.

Code for the Java Compiler tool:

```python
@tool  1 usage  new *
def java_compiler(code: str):
    """Compiles and Executes Java code."""
    with tempfile.TemporaryDirectory() as tmpdir:
        # Create a temporary Directory to run the code in and copy the code to a file inside it
        base_directory = os.path.dirname(os.path.abspath(__file__))
        java_bin = os.path.join(base_directory, "OpenJDK25U", "jdk-25.0.2+10", "bin")
        match = re.search( pattern: r'class\s+(\w+)', code)
        if match:
            class_name = match.group(1)
        else:
            class_name = "Main"
        student_file = os.path.join(tmpdir, f"{class_name}.java")
        with open(student_file, "w") as file:
            file.write(code)
        # Compile the code and run the code if it was successfully compiled. The output is then returned
        compiled_result = subprocess.run( args: [os.path.join(java_bin, "javac.exe"), student_file], capture_output=True, text=True, cwd=tmpdir)
        if compiled_result.returncode == 0:
            run_result = subprocess.run( args: [os.path.join(java_bin, "java.exe"), "-cp", tmpdir, class_name], capture_output=True, text=True, timeout=10)
            return run_result.stdout or run_result.stderr or "File doesn't Output Anything"
        # Error message is returned if the code wasn't successfully compiled
        else:
            return compiled_result.stdout or compiled_result.stderr or "Unknown Error."

# Java Compiler is added to tools that the LLM can use
tools = [java_compiler]
llm = ChatOllama(model="llama3.1:8b", temperature=0.0).bind_tools(tools, tool_choice="required")
```

In my graph I have 2 nodes. The first node is the call_model node. This node is responsible for looking at the code that is sent to it and deciding whether it needs to call the java_compiler tool or not to generate a response based on the system prompt given to it. I had to change the system prompt a lot because the model was being very stubborn and not doing what I was telling it to do. The prompt I ended up with is this: “You are a grading assistant." This ended up as my prompt because I couldn’t get the model to stop either not using and displaying the output the tool returned or guessing/hallucinating an answer for the output of the code no matter how I changed the prompt or what LLM I used. While I did try a few different LLMs, I am limited to ones with about 8 billion parameters or less since these are models that can fit on my GPU with 6GB of VRAM without needing to be offloaded, which massively increases the time it

takes for the model to run. I found this out the hard way by trying to run a 14 billion parameter model (qwen2.5-coder) and giving up after 15 minutes since it had still not finished running at that point. I decided that for now while I don't have the grading rubric functionality yet, I would store and print the tool's output directly as part of the chat message seen by the user to make sure that the tool's output is shown. The second node is the tool node, which is where the tool is called. The tool will then return the result after attempting to compile and run the code, which will either be its output or an error message of some kind. The graph's workflow is START with an edge that leads to the agent, then a conditional edge which leads to either the END node or to the tool node. The tool node then has an edge that loops back to the agent node. The condition edge's logic is handled by the should_continue function, which calls the tools node if a tool was called in the last message and the END node if a tool wasn't called in the last message.

Code for the LangGraph:

```python
# State Class is defined
class AgentState(TypedDict):  3+usages  new*
    # Add_messages is a specialized reducer that appends new messages to history
    messages: Annotated[list[BaseMessage], add_messages]
    # Output is used to store and display the output returned by the java_compiler tool
    output: AIMessage

def call_model(state: AgentState):  1usage  new*
    """
    The 'Thought' Node.
    The model examines the message history and decides whether
    to call a tool or provide a final answer.
    """
    messages = state['messages']
    # If tool was called before, the LLM's response and the output from the tool are returned
    for m in messages:
        if isinstance(m, ToolMessage):
            response = llm.invoke(messages)
            return {"messages": [response], "output": AIMessage(content=m.content)}
    # Inject system instructions if this is the start of the thread
    if len(messages) <= 1:
        system_prompt = SystemMessage(
            content= "You are a grading assistant."
        )
        messages = [system_prompt] + messages

    # Returns the LLM's response when the tool was not called
    response = llm.invoke(messages)
    return {"messages": [response], "output": AIMessage(content="")}

def should_continue(state: AgentState):  1usage  new*
    """
    The Conditional Edge.
    Checks if the last message contains tool calls.
    """
    last_message = state['messages'][-1]
    # If LLM requested a tool, route to the 'tools' node
    if last_message.tool_calls:
        return "tools"
    # If the LLM didn't request a tool, loop is finished
    return END

# Construct the Graph
workflow = StateGraph(AgentState)

# Define the two nodes in the cycle
workflow.add_node("agent", call_model)
workflow.add_node("tools", ToolNode(tools))

# Define the logic flow
workflow.set_entry_point("agent")
workflow.add_conditional_edges( source: "agent", should_continue)
workflow.add_edge( start_key: "tools",  end_key: "agent") # Loops back after tool execution

# Compile into a runnable application
app = workflow.compile()
```

To test the program I have 3 test files: one with a syntax error in the print statement, one that compiles and runs successfully and returns 2 numbers, and one that tries to check for a file that exists in the project directory. These are the results after uploading these files and giving them to the model.

Correct Code and Output (Java_Test2):

```java
public class Java_Test2 {
    public static void main(String[] args) {
        int num = 10;
        System.out.println("Number: " + num);
        for (int i = 1; i < 4; i++) {
            num = num * 2;
        }
        System.out.println("New Number: " + num);
    }
}
```

Java_Test2

```
C:\Users\desto\.jdks\openjdk-25\bin\java.exe "-javaagent:F:\JetBr
Number: 10
New Number: 80
```

Reset Chat History

Reset

AI Grading Assistant

Please Upload a Java File to be evaluated

Java_Test2.java 280 BB +

Evaluate Code

Files Uploaded:

Java_Test2.java

Output:

Number: 10 New Number: 80

In this code, we first print the initial value of `num`, which is 10. Then, in the for loop, we multiply `num` by 2 three times. This results in `num` being multiplied by 8 (2^3), so the final value of `num` is 80. Finally, we print this new value of `num`.

Files Uploaded:

Java_Test2.java

Incorrect Code and Output (Java_Test1):

```java
public class Java_Test1 {
    public static void main(String[] args) {
        int num = 10;
        System.out.println("Number: " + num);
        for (int i = 1; i < 4; i++) {
            num = num * num;
        }
        prit("New Number: " + num);
    }
}
```

```
: build failed At 4/ 2 sec, 486 ms
error
method prit(java.lang.String) :8

C:\Users\desto\IdeaProjects\ResearchProjectJavaFiles\src\Java_Test1.java:8:9
java: cannot find symbol
  symbol:   method prit(java.lang.String)
  location: class Java_Test1
```

Reset Chat History

Reset

Deploy

AI Grading Assistant

Please Upload a Java File to be evaluated

Java_Test1.java

Evaluate Code

Files Uploaded:

Java_Test1.java

Output:

C:\Users\desto\AppData\Local\Temp\tmp4kq44s8f.Java_Test1.java:1: error: cannot find symbol public class Java_Test1 { public static void main(String[] args) { int num = 10; System.out.println("Number: " + num); for (int i = 0; i < 3; i++) { num = num * 2; } prit("New Number: " + num); } } ^ symbol: method prit(String) location: class Java_Test1 1 error

The error message indicates that there is a typo in the code. The correct method name should be `print` instead of `prit`. Here's the corrected code:

```java
public class Java_Test1 {
    public static void main(String[] args) {
        int num = 10;
        System.out.println("Number: " + num);
        for (int i = 0; i < 3; i++) {
            num = num * 2;
        }
        System.out.print("New Number: " + num); // Note the use of print instead of prit
    }
}
```

This code will compile and run without any errors. The output will be:

```
Number: 10
New Number: 80
```

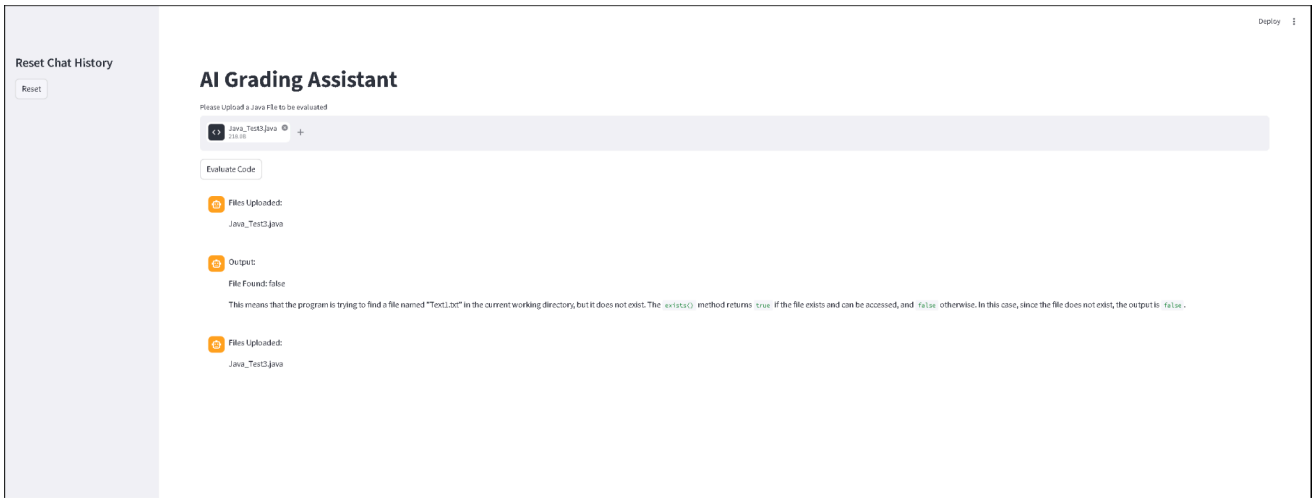
Files Uploaded:

Java_Test1.java

Search For FIle Code and Output (Java_Test3):

(Note for this test, the output from IntelliJ isn't shown with the code since it wouldn't be indicative of if the directory isolation works since it is being run from that project's directory.)

```java
import java.io.File;
public class Java_Test3 {
    public static void main(String[] args) {
        File test_file = new File( pathname: "Text1.txt");
        System.out.println("File Found: " + test_file.exists());
    }
}
```



In the output when given a file with code that compiles and runs, the chat message returned starts with the correct output that was returned by the tool of “Number: 10” and “New Number: 80”. The LLM’s response is then shown, which consists of breaking down the code step by step and explaining what happens at each step. For the output when given a file that doesn’t work, the output returned by the tool is an error message indicating that there is a syntax error in the code. The LLM then points out the error, explains it, gives a corrected version of the code, and finally what the model believes the output for that corrected code will be. Finally, for the output searching for a file in the project directory, the output is that the file was not found. This is good because this means that the temporary directory the code is compiled and run in is

isolated since the code that is being run in this temporary directory can't see the files that are in the project directory. This is followed by the LLM explaining why the code returns false.